LATIHAN TERSTRUKTUR
PERTEMUAN 4
IMPLEMENTASI UNIT TESTING DENGAN GITHUB ACTIONS
IMPLEMENTASI PENGUJIAN PERANGKAT LUNAK

Nama : Tia Anggraeni Sutisna

NIM : 3224004

Prodi : Sistem Informasi

Tanggal : 10 Oktober 2026

BAGIAN A — GIT & GITHUB DASAR

SOAL 1 — Git & GitHub Dasar

1. Perbedaan Centralized VCS dan Distributed VCS

Centralized Version Control System (CVCS) adalah sistem version control yang menggunakan satu server pusat sebagai tempat penyimpanan utama repository. Developer harus terhubung ke server tersebut untuk mengambil atau mengirim perubahan. Kekurangannya adalah ketika server pusat mengalami gangguan, proses kolaborasi juga dapat terganggu.

Contoh Centralized VCS:

1. Subversion atau SVN
2. CVS

Sedangkan Distributed Version Control System (DVCS) memungkinkan setiap developer memiliki salinan repository secara lengkap pada komputer masing-masing. Artinya, developer masih dapat melakukan beberapa aktivitas seperti commit dan melihat riwayat perubahan secara lokal tanpa harus selalu terhubung ke server.

Contoh Distributed VCS:

1. Git
2. Mercurial

Aspek	Centralized VCS	Distributed VCS
Repository	Terpusat pada satu server	Setiap pengguna memiliki salinan repository
Koneksi	Lebih bergantung pada server	Banyak operasi dapat dilakukan secara lokal
Risiko	Server pusat menjadi titik kegagalan	Repository tetap tersedia secara lokal
Contoh	SVN, CVS	Git, Mercurial

2. Lima Git Commands yang wajib dikuasai

Menurut saya, lima command Git dasar yang paling penting untuk dikuasai adalah:

Command	Fungsi	Contoh
git clone	Menyalin repository dari GitHub ke komputer lokal	git clone https://github.com/user/project.git
git add	Memasukkan perubahan file ke staging area	git add .

git commit	Menyimpan perubahan ke riwayat repository lokal	git commit -m "update project"
git push	Mengirim commit dari repository lokal ke GitHub	git push origin main
git pull	Mengambil perubahan terbaru dari remote repository	git pull origin main

3. Fungsi .gitignore

File .gitignore digunakan untuk menentukan file atau folder yang tidak ingin dimasukkan ke dalam version control Git.

Biasanya .gitignore digunakan untuk file yang bersifat sementara, hasil build, konfigurasi editor, atau file tertentu yang tidak perlu dibagikan ke repository.

Contohnya pada project Java Maven:

```
target/
.idea/
.vscode/
*.class
```

Dengan .gitignore, repository menjadi lebih bersih karena hanya file yang memang dibutuhkan project yang disimpan.

SOAL 2 — Praktik Git Commands

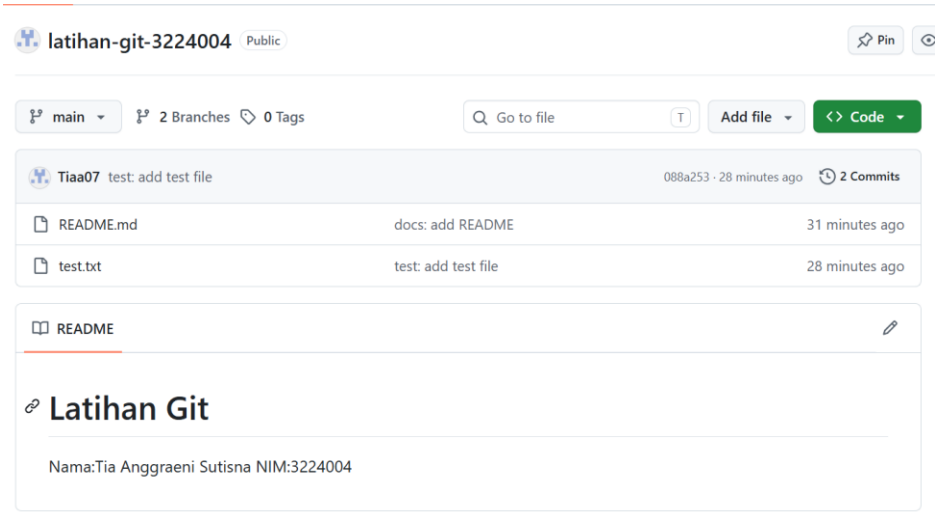
Pada praktik ini saya membuat repository baru di GitHub dengan nama latihan-git-3224004, kemudian melakukan proses clone repository ke komputer lokal, membuat file README.md, melakukan commit dan push, membuat branch feature-test, menambahkan file test.txt, serta melakukan merge kembali ke branch main.

Tabel Tahapan Pengerjaan

No.	Tahapan	Command yang Digunakan
1	Membuat repository baru di GitHub dengan nama latihan-git-3224004	Dilakukan melalui GitHub
2	Clone repository ke komputer lokal	git clone https://github.com/Tiaa07/latihan-git-3224004.git
3	Masuk ke folder repository	cd latihan-git-3224004

4	Membuat file README.md berisi nama dan NIM	echo # Latihan Git > README.md echo Nama: Tia Anggraeni Sutisna >> README.md echo NIM: 3224004 >> README.md
5	Menambahkan README.md ke staging area	git add README.md
6	Commit file README.md	git commit -m "docs: add README"
7	Push hasil commit ke branch main	git push origin main
8	Membuat branch baru feature-test	git branch feature-test
9	Berpindah ke branch feature-test	git switch feature-test
10	Membuat file test.txt	echo File ini digunakan untuk latihan branch feature-test. > test.txt
11	Menambahkan test.txt ke staging area	git add test.txt
12	Commit file test.txt	git commit -m "test: add test file"
13	Push branch feature-test ke GitHub	git push origin feature-test
14	Kembali ke branch main	git switch main
15	Merge branch feature-test ke main	git merge feature-test
16	Push hasil merge ke GitHub	git push origin main

Pesan commit dan nama branch tersebut mengikuti ketentuan yang ada di lembar kerja.



Berdasarkan praktik yang telah dilakukan, seluruh tahapan Git pada Soal 2 berhasil diselesaikan. Repository latihan-git-3224004 telah memiliki file README.md yang berisi nama dan NIM serta file test.txt yang sebelumnya dibuat pada branch feature-test. Branch tersebut kemudian berhasil di-merge ke branch main dan hasil akhirnya telah di-push ke GitHub.

BAGIAN B — UNIT TESTING FRAMEWORK

SOAL 3 — Konversi Test Case ke Kode Unit Test

1. Test Case Berdasarkan Independent Path

Pada Tugas Minggu 3, method validasiPendaftaran() telah dianalisis menggunakan pendekatan white-box testing untuk menentukan independent path. Pada Tugas Pertemuan 4 ini, sesuai instruksi yang meminta lima test case, pengujian dipilih untuk mewakili lima kondisi utama, yaitu validasi nama, umur, email, persetujuan syarat, serta kondisi ketika seluruh data valid.

a. Test Case 1 — Nama tidak diisi

Test case pertama digunakan untuk menguji kondisi ketika pengguna tidak mengisi nama.

Input yang digunakan:

```
nama = ""
umur = 20
email = "andi@gmail.com"
setujuSyarat = true
```

Expected result:

```
Nama harus diisi
```

Pada kondisi tersebut, program langsung berhenti pada validasi nama karena parameter nama berupa string kosong.

b. Test Case 2 — Umur kurang dari 17 tahun

Test case kedua digunakan untuk menguji batas minimal umur pendaftaran.

Input:

```
nama = "Andi"
umur = 16
email = "andi@gmail.com"
setujuSyarat = true
```

Expected result:

```
Umur minimal 17 tahun
```

Nama sudah memenuhi validasi pertama, tetapi nilai umur masih berada di bawah batas minimal 17 tahun sehingga program menghasilkan pesan tersebut.

c. **Test Case 3 — Email tidak valid**

Test case ketiga digunakan untuk menguji validasi format email. Pada pengujian ini email tidak memiliki karakter @.

Input:

```
nama = "Andi"
umur = 20
email = "andigmail.com"
setujuSyarat = true
```

Expected result:

```
Email tidak valid
```

Nama dan umur sudah valid, tetapi format email tidak memenuhi kondisi yang ditentukan program.

d. **Test Case 4 — Tidak menyetujui syarat dan ketentuan**

Test case keempat digunakan untuk menguji kondisi ketika seluruh data pengguna sudah valid tetapi pengguna belum menyetujui syarat dan ketentuan.

Input:

```
nama = "Andi"
umur = 20
email = "andi@gmail.com"
setujuSyarat = false
```

Expected result:

```
Harus menyetujui syarat dan ketentuan
```

Test case ini digunakan untuk memastikan bahwa pendaftaran tidak dapat dilanjutkan apabila pengguna belum menyetujui syarat dan ketentuan.

e. **Test Case 5 — Seluruh data valid**

Test case terakhir digunakan untuk menguji kondisi ketika seluruh input memenuhi persyaratan pendaftaran.

Input:

```
nama = "Andi"
umur = 20
email = "andi@gmail.com"
setujuSyarat = true
```

Expected result:

Pendaftaran berhasil

Pada kondisi tersebut seluruh proses validasi berhasil dilewati sehingga program mengembalikan pesan bahwa pendaftaran berhasil.

2. Implementasi Test Case ke Kode Unit Test JUnit

Kelima test case tersebut kemudian diimplementasikan menggunakan JUnit sebagai berikut:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

public class RegistrationValidatorTest {

    private final RegistrationValidator validator =
            new RegistrationValidator();

    @Test
    void testNamaKosong() {
        String hasil = validator.validasiPendaftaran(
                "", 20, "andi@gmail.com", true);

        assertEquals("Nama harus diisi", hasil);
    }

    @Test
    void testUmurKurangDari17() {
        String hasil = validator.validasiPendaftaran(
                "Andi", 16, "andi@gmail.com", true);

        assertEquals("Umur minimal 17 tahun", hasil);
    }

    @Test
    void testEmailTidakValid() {
```

```
        String hasil = validator.validasiPendaftaran(
                "Andi", 20, "andigmail.com", true);

        assertEquals("Email tidak valid", hasil);
    }

    @Test
    void testTidakMenyetujuiSyarat() {
        String hasil = validator.validasiPendaftaran(
                "Andi", 20, "andi@gmail.com", false);

        assertEquals(
                "Harus menyetujui syarat dan ketentuan",
                hasil);
    }

    @Test
    void testPendaftaranBerhasil() {
        String hasil = validator.validasiPendaftaran(
                "Andi", 20, "andi@gmail.com", true);

        assertEquals("Pendaftaran berhasil", hasil);
    }
}
```

Penjelasan Kode Unit Test

Pada setiap method pengujian digunakan anotasi:

```
@Test
```

yang menandakan bahwa method tersebut merupakan unit test.

Kemudian method:

```
assertEquals()
```

digunakan untuk membandingkan hasil yang diharapkan dengan hasil aktual dari method validasiPendaftaran().

Sebagai contoh:

```
assertEquals("Email tidak valid", hasil);
```

berarti unit test akan dinyatakan berhasil apabila output aktual dari program sama dengan "Email tidak valid".

Kalau hasil aktual berbeda dari expected result, maka test akan dinyatakan gagal.

SOAL 4 — Implementasi Unit Test StudentGrade

Kode pada lembar kerja:

```
public class StudentGrade {

    public String calculateGrade(int score) {
        if (score < 0 || score > 100) {
            throw new IllegalArgumentException(
                "Score must be between 0 and 100"
            );
        }

        if (score >= 80) {
            return "A";
        } else if (score >= 70) {
            return "B";
        } else if (score >= 60) {
            return "C";
        } else if (score >= 50) {
            return "D";
        } else {
            return "E";
        }
    }

    public boolean isPassed(int score) {
        if (score < 0 || score > 100) {
            throw new IllegalArgumentException(
```

```
            "Score must be between 0 and 100"
        );
    }

    return score >= 60;
  }
}
```

1. Menghitung Cyclomatic Complexity Method calculateGrade()

Cyclomatic Complexity digunakan untuk mengetahui jumlah jalur independen yang terdapat dalam suatu program. Pada method calculateGrade(), terdapat enam predicate yang memengaruhi alur program, yaitu:

P1 = score < 0
P2 = score > 100
P3 = score >= 80
P4 = score >= 70
P5 = score >= 60
P6 = score >= 50

Kondisi:

```
if (score < 0 || score > 100)
```

Pada analisis ini digunakan pendekatan short-circuit evaluation pada operator ||. Oleh karena itu, kondisi score < 0 dan score > 100 diperlakukan sebagai dua predicate/decision yang terpisah karena kondisi kedua hanya dievaluasi ketika kondisi pertama bernilai false.

Rumus yang digunakan:

$$V(G) = P + 1$$

dengan P adalah jumlah decision/predicate.

Jumlah decision:

$$P = 6$$

Sehingga:

$$V(G) = 6 + 1$$
$$V(G) = 7$$

Jadi, nilai Cyclomatic Complexity pada method calculateGrade() adalah 7. Artinya, terdapat tujuh independent path yang dapat digunakan untuk mewakili kemungkinan jalur eksekusi utama pada method tersebut.

2. Menentukan Independent Path Method calculateGrade()

Berdasarkan nilai Cyclomatic Complexity sebesar 7, maka terdapat tujuh independent path yang dianalisis.

a. Path 1 — Score kurang dari 0

Contoh input:

score = -1

Kondisi:

score < 0 = True

Jalur:

Start **→ score < 0 = True** **→ IllegalArgumentException** **→ End**

Expected result:

IllegalArgumentException

dengan pesan:

Score must be between 0 and 100

b. Path 2 — Score lebih dari 100

Contoh input:

score = 101

Kondisi:

score < 0 = False **score > 100 = True**

Jalur:

Start **→ score < 0 = False** **→ score > 100 = True** **→ IllegalArgumentException** **→ End**

Expected result:

IllegalArgumentException

dengan pesan:

Score must be between 0 and 100

c. **Path 3 — Grade A**

Contoh input:

score = 85

Kondisi:

score < 0 = False

score > 100 = False

score >= 80 = True

Jalur:

Start

→ score valid

→ score >= 80 = True

→ return "A"

→ End

Expected result:

A

d. **Path 4 — Grade B**

Contoh:

score = 75

Kondisi:

score >= 80 = False

score >= 70 = True

Jalur:

Start

→ score valid

→ score >= 80 = False

→ score >= 70 = True

→ return "B"

→ End

Expected:

B

e. **Path 5 — Grade C**

Contoh:

score = 65

Kondisi:

score >= 80 = False **score >= 70 = False** **score >= 60 = True**

Jalur:

Start **→ score valid** **→ score >= 80 = False** **→ score >= 70 = False** **→ score >= 60 = True** **→ return "C"** **→ End**

Expected result:

C

f. **Path 6 — Grade D**

Contoh:

score = 55

Kondisi:

score >= 80 = False **score >= 70 = False** **score >= 60 = False** **score >= 50 = True**

Jalur:

Start **→ score valid** **→ score >= 80 = False** **→ score >= 70 = False** **→ score >= 60 = False**

→ **score >= 50 = True**

→ **return "D"**

→ **End**

Expected result:

D

g. **Path 7 — Grade E**

Contoh:

score = 40

Kondisi:

score >= 80 = False

score >= 70 = False

score >= 60 = False

score >= 50 = False

Jalur:

Start

→ **score valid**

→ **score >= 80 = False**

→ **score >= 70 = False**

→ **score >= 60 = False**

→ **score >= 50 = False**

→ **return "E"**

→ **End**

Expected:

E

Ringkasan Independent Path

Path	Input	Kondisi Utama	Expected Result
Path 1	-1	score < 0	IllegalArgumentException
Path 2	101	score > 100	IllegalArgumentException
Path 3	85	score >= 80	A
Path 4	75	score >= 70	B
Path 5	65	score >= 60	C
Path 6	55	score >= 50	D
Path 7	40	score < 50	E

3. Implementasi Unit Test JUnit untuk Method calculateGrade() dan isPassed()

Unit test dibuat untuk menguji dua method yang terdapat pada class StudentGrade, yaitu calculateGrade() dan isPassed(). Pengujian pada method calculateGrade() mencakup setiap kemungkinan hasil grade serta kedua kondisi nilai tidak valid, yaitu score kurang dari 0 dan score lebih dari 100. Sedangkan pengujian isPassed() dilakukan terhadap kondisi lulus, batas minimal kelulusan, tidak lulus, serta input tidak valid. Soal memang meminta implementasi JUnit untuk kedua method tersebut.

Kode unit test JUnit yang digunakan adalah sebagai berikut:

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class StudentGradeTest {

   private final StudentGrade studentGrade =
         new StudentGrade();

   // Pengujian method calculateGrade()

   @Test
      void testScoreKurangDariNol() {
         assertThrows(
               IllegalArgumentException.class,
               () -> studentGrade.calculateGrade(-1)
         );
      }

   @Test
      void testScoreLebihDari100() {
         assertThrows(
               IllegalArgumentException.class,
               () -> studentGrade.calculateGrade(101)
         );
      }
```

```
@Test
void testGradeA() {
   assertEquals("A",
         studentGrade.calculateGrade(85));
}

@Test
void testGradeB() {
   assertEquals("B",
         studentGrade.calculateGrade(75));
}

@Test
void testGradeC() {
   assertEquals("C",
         studentGrade.calculateGrade(65));
}

@Test
void testGradeD() {
   assertEquals("D",
         studentGrade.calculateGrade(55));
}

@Test
void testGradeE() {
   assertEquals("E",
         studentGrade.calculateGrade(40));
}

// Pengujian method isPassed()
```

```
    @Test
    void testLulus() {
        assertTrue(studentGrade.isPassed(70));
    }

    @Test
    void testBatasMinimalLulus() {
        assertTrue(studentGrade.isPassed(60));
    }

    @Test
    void testTidakLulus() {
        assertFalse(studentGrade.isPassed(59));
    }

    @Test
    void testScoreTidakValidPadaIsPassed() {
        assertThrows(
                IllegalArgumentException.class,
                () -> studentGrade.isPassed(101)
        );
    }
}
```

Penjelasan Unit Test

Pada method calculateGrade(), pengujian dilakukan terhadap tujuh kondisi utama. Dua pengujian pertama digunakan untuk memastikan bahwa score di luar rentang 0–100 menghasilkan IllegalArgumentException. Pengujian berikutnya memastikan bahwa setiap rentang nilai menghasilkan grade A, B, C, D, atau E sesuai ketentuan program.

Method:

assertThrows()

digunakan untuk memastikan bahwa program menghasilkan exception ketika input tidak valid.

Method:

assertEquals()

digunakan untuk membandingkan grade aktual yang dihasilkan program dengan grade yang diharapkan.

Pada method isPassed(), pengujian dilakukan terhadap nilai lulus, nilai batas minimal kelulusan, nilai tidak lulus, dan nilai tidak valid. Nilai 60 dan 59 digunakan untuk menguji batas kelulusan karena method isPassed() menetapkan bahwa mahasiswa dinyatakan lulus apabila:

score >= 60

Method:

assertTrue()

digunakan untuk kondisi yang diharapkan menghasilkan nilai true, sedangkan:

assertFalse()

digunakan untuk kondisi yang diharapkan menghasilkan nilai false.

Ringkasan Pengujian JUnit

Pengujian	Input	Expected Result
Score kurang dari 0	-1	IllegalArgumentException
Score lebih dari 100	101	IllegalArgumentException
Grade A	85	"A"
Grade B	75	"B"
Grade C	65	"C"
Grade D	55	"D"
Grade E	40	"E"
Lulus	70	true
Batas lulus	60	true
Tidak lulus	59	false
Invalid isPassed()	101	IllegalArgumentException

BAGIAN C — GITHUB ACTIONS

SOAL 5 — Analisis Workflow YAML

1. Apa nama workflow di atas?

Nama workflow tersebut adalah:

```
Python CI
```

Nama tersebut ditentukan pada bagian:

```
name: Python CI
```

Jadi, Python CI merupakan nama workflow yang akan ditampilkan pada halaman GitHub Actions.

2. Trigger/event apa saja yang memicu workflow?

Workflow dijalankan oleh dua jenis event, yaitu push dan pull_request.

Konfigurasinya adalah:

```
on:
  push:
    branches: [ main, develop ]

  pull_request:
    branches: [ main ]
```

Artinya:

- Workflow akan berjalan ketika terjadi push ke branch main
- Workflow akan berjalan ketika terjadi push ke branch develop
- Workflow juga akan berjalan ketika ada pull request menuju branch main

3. Apa fungsi dari strategy matrix?

Pada workflow terdapat konfigurasi:

```
strategy:
  matrix:
    python-version: ['3.10', '3.11', '3.12']
```

Strategy matrix digunakan untuk menjalankan job yang sama pada beberapa konfigurasi secara otomatis. Pada workflow tersebut, pengujian dijalankan menggunakan Python versi 3.10, 3.11, dan 3.12. Dengan adanya matrix, developer tidak perlu membuat job terpisah untuk setiap versi

Python. Hal ini juga membantu memastikan bahwa project dapat berjalan dan lulus pengujian pada beberapa versi Python yang berbeda.

4. Sebutkan 5 steps yang ada di dalam job test

Berdasarkan workflow YAML yang diberikan, terdapat empat steps yang terlihat di dalam job test, yaitu Checkout code, Setup Python, Install dependencies, dan Run tests. Walaupun pada pertanyaan tertulis "sebutkan 5 steps", pada konfigurasi YAML yang tersedia hanya terdapat empat steps.

Empat steps tersebut adalah:

Step 1 — Checkout code

```
- name: Checkout code
  uses: actions/checkout@v4
```

Fungsinya untuk mengambil source code repository agar dapat digunakan oleh job selanjutnya.

Step 2 — Setup Python

```
- name: Setup Python ${{ matrix.python-version }}
  uses: actions/setup-python@v5
  with:
    python-version: ${{ matrix.python-version }}
```

Fungsinya untuk menyiapkan versi Python sesuai dengan matrix, yaitu 3.10, 3.11, dan 3.12.

Step 3 — Install dependencies

```
- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install pytest pytest-cov
```

Fungsinya untuk memperbarui pip dan menginstal dependency yang dibutuhkan untuk pengujian, yaitu pytest dan pytest-cov.

Step 4 — Run tests

```
- name: Run tests
  run: pytest --cov=src --cov-report=xml
```

Fungsinya untuk menjalankan unit test menggunakan pytest sekaligus menghasilkan laporan code coverage untuk folder src.

No.	Step	Fungsi
1	Checkout code	Mengambil source code repository

2	Setup Python	Menyiapkan Python sesuai versi matrix
3	Install dependencies	Menginstal pytest dan pytest-cov
4	Run tests	Menjalankan test dan menghasilkan coverage

SOAL 6 — Membuat Workflow YAML

Pada Soal 6 dibuat workflow GitHub Actions untuk menjalankan unit test secara otomatis pada project Java. Sesuai ketentuan pada lembar kerja, workflow diberi nama Java Unit Test, menggunakan runner ubuntu-latest, dijalankan ketika terjadi push ke branch main atau develop, serta ketika terdapat pull_request menuju branch main. Workflow juga menjalankan unit test menggunakan Maven, menghasilkan laporan code coverage menggunakan JaCoCo, dan mengunggah hasil coverage sebagai artifact dengan nama jacoco-report.

1. Implementasi Workflow GitHub Actions

File workflow dibuat dengan nama:

java-unit-test.yml

dan ditempatkan pada:

.github/workflows/java-unit-test.yml

Kode YAML yang digunakan adalah sebagai berikut:

```yaml
name: Java Unit Test

on:
  push:
    branches: [ main, develop ]

  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

```
- name: Setup JDK 17
  uses: actions/setup-java@v4
  with:
    distribution: 'temurin'
    java-version: '17'

- name: Run tests
  working-directory: Tugas/Tugas04
  run: mvn test

- name: Generate coverage report
  working-directory: Tugas/Tugas04
  run: mvn jacoco:report

- name: Upload artifact
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-report
    path: Tugas/Tugas04/target/site/jacoco/
```

2. Penjelasan Workflow

a. Nama Workflow

```
name: Java Unit Test
```

Bagian tersebut digunakan untuk memberikan nama pada workflow. Nama Java Unit Test akan ditampilkan pada halaman Actions di repository GitHub.

b. Trigger Workflow

```
on:
  push:
    branches: [ main, develop ]

  pull_request:
    branches: [ main ]
```

Konfigurasi tersebut menentukan kapan workflow dijalankan. Workflow akan berjalan ketika terdapat push ke branch main atau develop, serta ketika terdapat pull_request menuju branch main.

c. Runner

```
runs-on: ubuntu-latest
```

Runner ubuntu-latest digunakan sebagai lingkungan tempat job test dijalankan.

d. Checkout Code

```
- name: Checkout code
  uses: actions/checkout@v4
```

Step ini digunakan untuk mengambil source code dari repository agar dapat digunakan pada proses selanjutnya.

e. Setup JDK 17

```
- name: Setup JDK 17
  uses: actions/setup-java@v4
  with:
    distribution: 'temurin'
    java-version: '17'
```

Step ini digunakan untuk menyiapkan Java Development Kit versi 17 sesuai dengan ketentuan pada lembar kerja.

f. Menjalankan Unit Test

```
- name: Run tests
  working-directory: Tugas/Tugas04
  run: mvn test
```

Perintah:

```
mvn test
```

digunakan untuk menjalankan seluruh unit test pada project Java menggunakan Maven. working-directory: Tugas/Tugas04 digunakan karena file pom.xml dan source code project berada di dalam folder Tugas/Tugas04, bukan langsung pada root repository.

g. Generate Coverage Report

```
- name: Generate coverage report
  working-directory: Tugas/Tugas04
  run: mvn jacoco:report
```

Perintah:

```
mvn jacoco:report
```

digunakan untuk menghasilkan laporan code coverage menggunakan JaCoCo.

h. Upload Artifact

```
- name: Upload artifact
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-report
    path: Tugas/Tugas04/target/site/jacoco/
```

Step terakhir digunakan untuk mengunggah hasil laporan code coverage sebagai artifact dengan nama:

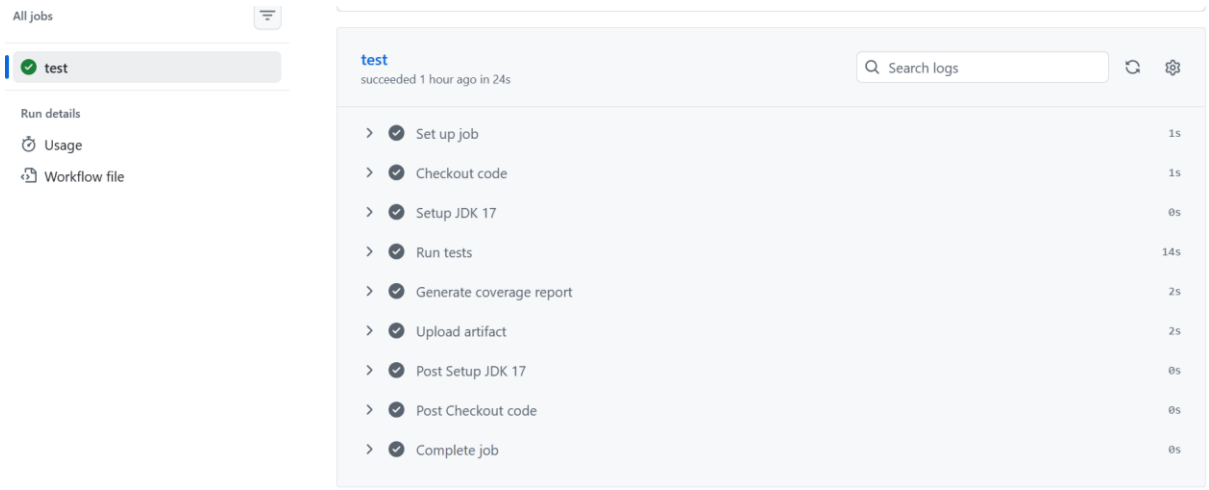
```
jacoco-report
```

Lokasi report yang diunggah adalah:

```
Tugas/Tugas04/target/site/jacoco/
```

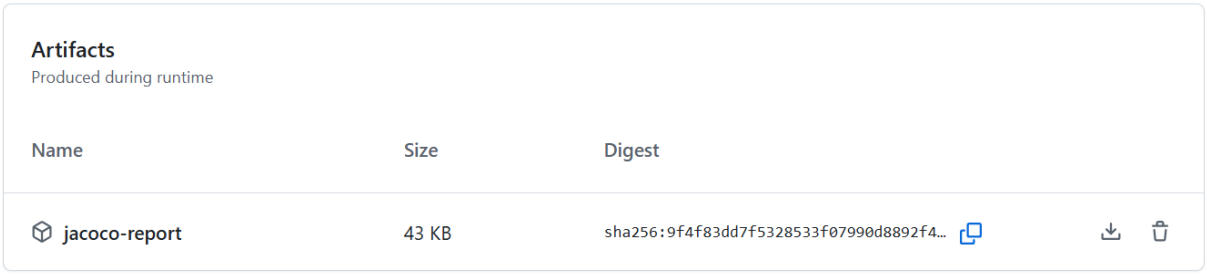
3. Hasil Implementasi GitHub Actions

Setelah file workflow di-push ke branch main, GitHub Actions secara otomatis menjalankan workflow Java Unit Test. Berdasarkan hasil pengujian, job test berhasil dijalankan dan seluruh step utama memperoleh status berhasil.



Berdasarkan hasil workflow tersebut, seluruh tahapan pengujian berhasil dijalankan. Source code berhasil di-checkout, JDK 17 berhasil disiapkan, unit test berhasil dijalankan, coverage report berhasil dibuat, dan artifact berhasil diunggah.

4. Hasil Upload Artifact JaCoCo



Artifact jacoco-report menunjukkan bahwa proses pembuatan laporan code coverage menggunakan JaCoCo berhasil dilakukan. Artifact tersebut berisi hasil coverage report yang dihasilkan selama workflow GitHub Actions berlangsung.

BAGIAN D — CODE COVERAGE & INTEGRASI

SOAL 7 — Analisis Code Coverage

Berdasarkan coverage report yang diberikan, terdapat empat file yang dianalisis, yaitu calculator.py, login_system.py, payment.py, dan utils.py. Setiap file memiliki jumlah statement, jumlah statement yang tidak tereksekusi (Miss), serta persentase coverage yang berbeda.

1. Menghitung Total Statement pada Project

Jumlah seluruh statement diperoleh dengan menjumlahkan statement dari setiap file:

calculator.py = 10
login_system.py = 15
payment.py = 20
utils.py = 5

Perhitungannya:

Total statement = 10 + 15 + 20 + 5
= 50

Jadi, total statement pada project adalah 50 statement.

2. Menghitung Total Statement yang Tidak Tereksekusi

Jumlah statement yang tidak tereksekusi diperoleh dari kolom Miss:

calculator.py = 0
login_system.py = 2
payment.py = 8
utils.py = 1

Perhitungannya:

Total Miss = 0 + 2 + 8 + 1
= 11

Jadi, total statement yang tidak tereksekusi adalah 11 statement.

3. Menentukan File dengan Coverage Paling Rendah

Berdasarkan laporan coverage:

calculator.py = 100%
login_system.py = 87%
payment.py = 60%

utils.py = 80%

File dengan coverage paling rendah adalah:

payment.py

dengan coverage:

60%

Jadi, file payment.py memiliki coverage paling rendah, yaitu 60%.

4. Upaya Meningkatkan Coverage pada payment.py

Coverage payment.py hanya 60% karena masih terdapat 8 statement yang belum tereksekusi dari total 20 statement.

Untuk meningkatkan coverage, perlu dilakukan penambahan unit test yang mengeksekusi bagian kode yang belum tercakup. Pengujian sebaiknya tidak hanya berfokus pada alur normal, tetapi juga mencakup kondisi lain seperti percabangan, input tidak valid, error handling, dan kemungkinan edge case yang ada pada file payment.py.

Dengan menambahkan test case yang mencakup statement yang sebelumnya belum dieksekusi, jumlah Miss dapat dikurangi sehingga persentase coverage meningkat.

5. Apakah Coverage 78% Sudah Cukup Baik?

Menurut saya, coverage 78% sudah cukup baik sebagai kondisi awal, karena sebagian besar statement pada project sudah dieksekusi oleh test. Namun, nilai tersebut belum dapat dianggap optimal karena masih terdapat 11 statement yang belum tercakup oleh pengujian.

Selain itu, nilai coverage total dapat menutupi kelemahan pada file tertentu. Contohnya, meskipun total coverage project mencapai 78%, file payment.py hanya memiliki coverage 60%. Oleh karena itu, evaluasi sebaiknya tidak hanya melihat coverage total, tetapi juga coverage setiap file dan bagian kode yang penting.

Coverage yang tinggi juga tidak secara otomatis menjamin bahwa seluruh logika program sudah diuji dengan baik. Kualitas test case tetap perlu diperhatikan agar pengujian tidak hanya mengeksekusi kode, tetapi juga memverifikasi hasil program secara benar.

BAGIAN E — STUDI KASUS TERPADU

SOAL 8 — Implementasi Lengkap BankAccount

Pada Soal 8 dilakukan implementasi studi kasus terpadu menggunakan class BankAccount. Class tersebut memiliki method deposit(double amount), withdraw(double amount), dan getBalance(). Selain implementasi program, dibuat juga unit test menggunakan JUnit, workflow GitHub Actions, serta laporan code coverage menggunakan JaCoCo. Soal ini bertujuan untuk menggabungkan proses pengembangan kode, unit testing, continuous integration, dan code coverage dalam satu project.

1. Implementasi Class BankAccount

Class BankAccount digunakan untuk merepresentasikan rekening sederhana yang memiliki saldo. Saldo awal diatur sebesar 0.0. Method deposit() digunakan untuk menambahkan saldo, sedangkan method withdraw() digunakan untuk mengurangi saldo. Method getBalance() digunakan untuk memperoleh saldo saat ini.

Kode yang digunakan adalah sebagai berikut:

```
public class BankAccount {

    private double balance;

    public BankAccount() {
        this.balance = 0.0;
    }

    public void deposit(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException(
                    "Amount tidak boleh negatif"
            );
        }

        balance += amount;
    }
```

```
    public void withdraw(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException(
                    "Amount tidak boleh negatif"
            );
        }

        if (amount > balance) {
            throw new IllegalArgumentException(
                    "Saldo tidak mencukupi"
            );
        }

        balance -= amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

Berdasarkan kode tersebut, terdapat dua validasi utama. Pertama, nilai amount tidak boleh negatif. Kedua, proses withdraw() tidak diperbolehkan apabila jumlah penarikan melebihi saldo yang tersedia. Kedua kondisi tersebut sesuai dengan ketentuan pada studi kasus.

2. Implementasi Unit Test BankAccount

Unit test dibuat menggunakan JUnit untuk menguji seluruh method pada class BankAccount. Sesuai ketentuan soal, jumlah unit test yang dibuat minimal enam test case. Pada implementasi ini digunakan tujuh test case.

Kode unit test yang digunakan adalah sebagai berikut:

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
```

```
public class BankAccountTest {

    private BankAccount account;

    @BeforeEach
    void setUp() {
        account = new BankAccount();
    }

    @Test
    void testSaldoAwal() {
        assertEquals(0.0, account.getBalance());
    }

    @Test
    void testDepositBerhasil() {
        account.deposit(100000);

        assertEquals(100000.0, account.getBalance());
    }

    @Test
    void testDepositNegatif() {
        assertThrows(
                IllegalArgumentException.class,
                () -> account.deposit(-50000)
        );
    }

    @Test
    void testWithdrawBerhasil() {
        account.deposit(100000);
        account.withdraw(40000);
```

```
        assertEquals(60000.0, account.getBalance());
    }

    @Test
    void testWithdrawMelebihiSaldo() {
        account.deposit(50000);

        assertThrows(
                IllegalArgumentException.class,
                () -> account.withdraw(60000)
        );
    }

    @Test
    void testWithdrawNegatif() {
        account.deposit(50000);

        assertThrows(
                IllegalArgumentException.class,
                () -> account.withdraw(-10000)
        );
    }

    @Test
    void testGetBalanceSetelahBeberapaTransaksi() {
        account.deposit(100000);
        account.deposit(50000);
        account.withdraw(25000);

        assertEquals(125000.0, account.getBalance());
    }
}
```

Ketujuh test case tersebut digunakan untuk menguji kondisi saldo awal, deposit berhasil, deposit negatif, withdraw berhasil, withdraw melebihi saldo, withdraw negatif, serta nilai saldo setelah beberapa transaksi.

Ringkasan pengujian dapat ditulis seperti ini:

No.	Test Case	Input/Kondisi	Expected Result
1	Saldo awal	Akun baru	0.0
2	Deposit berhasil	Deposit 100000	Saldo 100000.0
3	Deposit negatif	Deposit -50000	IllegalArgumentException
4	Withdraw berhasil	Saldo 100000, withdraw 40000	Saldo 60000.0
5	Withdraw melebihi saldo	Saldo 50000, withdraw 60000	IllegalArgumentException
6	Withdraw negatif	Withdraw -10000	IllegalArgumentException
7	Get balance setelah transaksi	Deposit 100000, deposit 50000, withdraw 25000	Saldo 125000.0

3. Implementasi GitHub Actions

Workflow GitHub Actions yang digunakan pada Soal 6 juga digunakan untuk menjalankan unit test BankAccountTest. Workflow dijalankan secara otomatis ketika terdapat push ke branch main atau develop, serta ketika terdapat pull_request menuju branch main.

Workflow menjalankan unit test menggunakan Maven melalui perintah:

```
mvn test
```

dan menghasilkan laporan code coverage menggunakan:

```
mvn jacoco:report
```

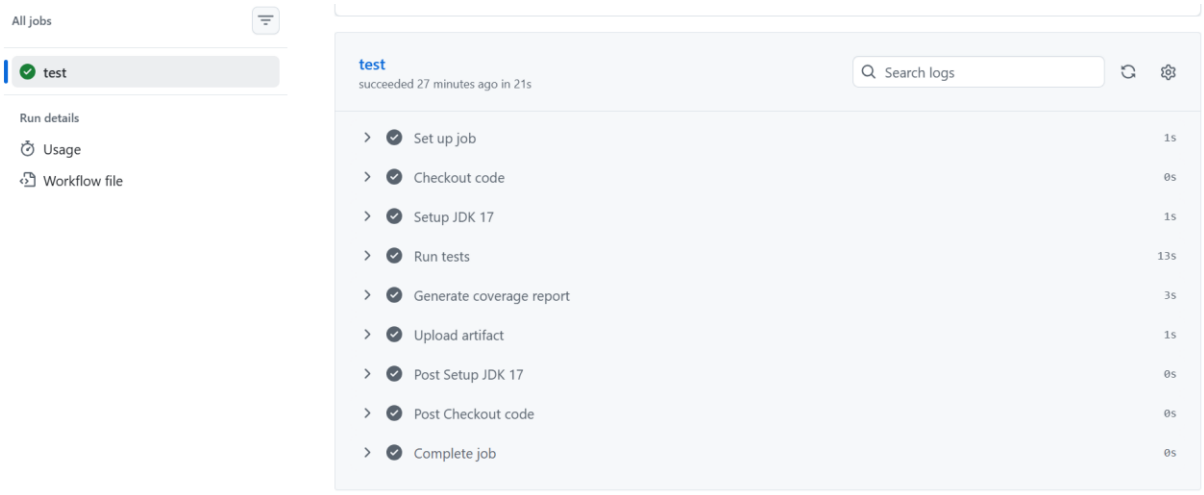
Hasil coverage kemudian diunggah sebagai artifact dengan nama:

```
jacoco-report
```

Dengan demikian, unit test BankAccountTest ikut dijalankan bersama unit test lain yang terdapat dalam project.

4. Hasil Pipeline GitHub Actions

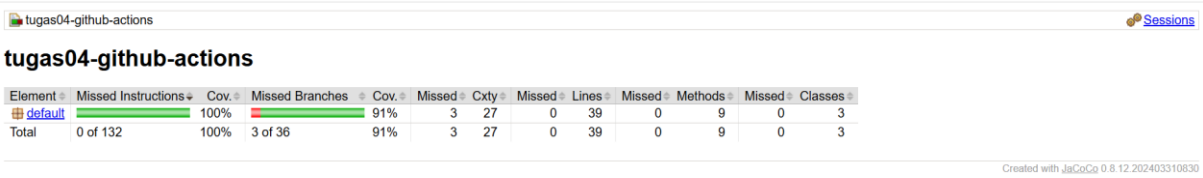
Setelah perubahan pada class BankAccount dan BankAccountTest di-push ke repository GitHub, workflow Java Unit Test dijalankan secara otomatis.



Berdasarkan hasil workflow tersebut, proses checkout source code, setup JDK 17, unit testing, pembuatan coverage report, dan upload artifact berhasil dijalankan tanpa error. Hal ini menunjukkan bahwa implementasi BankAccount dan unit test yang dibuat berhasil dijalankan melalui pipeline GitHub Actions.

5. Hasil Code Coverage JaCoCo

Laporan code coverage dihasilkan menggunakan JaCoCo setelah seluruh unit test selesai dijalankan.



Laporan JaCoCo digunakan untuk melihat seberapa besar bagian kode yang telah dieksekusi oleh unit test. Melalui laporan tersebut dapat diketahui coverage pada instruction, branch, line, method, dan class yang terdapat dalam project. Class BankAccount juga tercakup dalam laporan coverage sehingga dapat diketahui bagian kode yang telah diuji oleh unit test.

6. Link Repository GitHub

Link Repository GitHub: **https://github.com/Tiaa07/latihan-git-3224004**